

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático 3 - Gerenciador de Escopos com Pilha

BCC202 - Estrutura de Dados I

Ana Júlia Barros Araújo
Professor: Nome do professor

Ouro Preto
21 de fevereiro de 2026

Sumário

1	Introdução	1
1.1	Especificações do problema	1
1.2	Considerações iniciais	1
1.3	Ferramentas utilizadas	1
1.4	Especificações da máquina	1
1.5	Instruções de compilação e execução	2
2	Desenvolvimento	3
2.1	Estruturas de Dados	3
2.2	Funções Principais	3
2.2.1	Gerenciamento da Árvore Binária	3
2.2.2	Gerenciamento da Pilha de Escopos	4
2.2.3	Processamento das Linhas	4
2.3	Decisões de Implementação	5
3	Experimentos	6
4	Resultados	7
4.1	Exemplos de Execução	7
4.2	Análise de Memória	7
4.3	Pontos Fortes e Fracos	7
5	Considerações finais	8

Lista de Códigos Fonte

1	Estruturas de dados principais	3
2	Funções de inserção e busca na árvore	3
3	Busca de variável na pilha de escopos	4
4	Processamento dos comandos	5

1 Introdução

PEste trabalho consiste no desenvolvimento de um interpretador simples para gerenciamento de escopos de variáveis utilizando uma estrutura de pilha em linguagem C. O programa é capaz de ler um arquivo de entrada contendo comandos de criação de escopos, declaração de variáveis e impressão de valores, implementando uma árvore binária de busca para armazenar as variáveis em cada escopo. A codificação foi feita em C utilizando somente a biblioteca padrão da GNU, sem o uso de bibliotecas adicionais.

1.1 Especificações do problema

O problema consiste em implementar um sistema que gerencie variáveis em diferentes escopos aninhados. O programa deve ler um arquivo de entrada com a seguinte sintaxe:

- *begin* - inicia um novo escopo
- *end* - finaliza o escopo atual
- *var nome = valor* - declara uma variável no escopo atual
- *print nome* - imprime o valor de uma variável (buscando nos escopos encadeados)

A implementação deve utilizar uma pilha para gerenciar os escopos e, dentro de cada escopo, uma árvore binária de busca para armazenar as variáveis.

1.2 Considerações iniciais

Durante a criação deste projeto, foram utilizadas as seguintes ferramentas:

- Ambiente de desenvolvimento do código fonte: Visual Studio Code.¹
- Linguagem utilizada: C.
- Ambiente de desenvolvimento da documentação: Overleaf L^AT_EX.²

1.3 Ferramentas utilizadas

Para testar e validar a implementação, foram utilizadas as seguintes ferramentas:

- *GCC*: ferramentas de análise estática do código.
- *Valgrind*: ferramentas de análise dinâmica do código.

1.4 Especificações da máquina

A máquina onde o desenvolvimento e os testes foram realizados possui a seguinte configuração:

- *Processador*: Intel Core I5.
- *Memória RAM*: 8 Gb.
- *Sistema Operacional*: Ubuntu 24.04.

¹Visual Studio Code está disponível em <https://code.visualstudio.com>

²Disponível em <https://www.overleaf.com/>

1.5 Instruções de compilação e execução

Para a compilação do projeto, basta digitar no terminal:

Compilando o projeto

```
gcc -c filaprocessos.c -Wall gcc -c tp.c -Wall gcc filaprocessos.o tp.o -o exe
```

Usou-se para a compilação as seguintes opções:

- *-c*: traduz o arquivo *.c* em um arquivo objeto (*.o*)
- *-o*: indica que o termo seguinte é o nome do arquivo executável
- *-Wall*: para mostrar todos os possível *warnings* do código.

Para a execução do programa basta digitar:

```
./exe arquivodeentrada
```

Onde *arquivo_{de}entrada* é o caminho para o arquivo contendo os comandos a serem interpretados.

2 Desenvolvimento

A solução implementada utiliza duas estruturas de dados principais: uma pilha para gerenciar os escopos e, dentro de cada escopo, uma árvore binária de busca para armazenar as variáveis. Esta abordagem permite o aninhamento de escopos e a busca por variáveis seguindo a regra de escopos léxico (busca do escopo mais interno para o mais externo).

2.1 Estruturas de Dados

Foram definidas as seguintes estruturas:

- **No**: representa um nó da árvore binária, contendo nome da variável, valor e ponteiros para os filhos esquerdo e direito
- **Escopo**: contém um ponteiro para a raiz da árvore binária do escopo
- **Elemento**: elemento da pilha, contém um ponteiro para um escopo e um ponteiro para o elemento anterior
- **Pilha**: gerencia a pilha de escopos, com ponteiro para o topo e controle de tamanho

```
1 typedef struct no{
2     char nome[MAX_VAR];
3     char valor[MAX_VAR];
4     struct no *pEsq;
5     struct no *pDir;
6 } No;
7
8 typedef struct{
9     No *pRaiz;
10 } Escopo;
11
12 typedef struct elemento{
13     Escopo *escopo;
14     struct elemento *pAnterior;
15 } Elemento;
16
17 typedef struct{
18     Elemento *pTopo;
19     int tamanho;
20 } Pilha;
```

Código 1: Estruturas de dados principais

2.2 Funções Principais

2.2.1 Gerenciamento da Árvore Binária

As funções para manipulação da árvore binária implementam as operações básicas de inserção e busca:

- **criarNo**: aloca memória e inicializa um nó com nome e valor
- **inserirNo**: insere um nó na árvore de forma ordenada (pelo nome da variável)
- **destroiEscopo**: libera recursivamente toda a memória alocada para a árvore

```
1 No *inserirNo(No *no, const char *nome, const char *valor){
2     if(no == NULL){
3         return criarNo(nome, valor);
4     }
5
6     int compare = strcmp(nome, no->nome);
```

```

7     if(compare < 0){
8         no->pEsq = inserirNo(no->pEsq, nome, valor);
9     } else if(compare > 0){
10        no->pDir = inserirNo(no->pDir, nome, valor);
11    } else{
12        strcpy(no->valor, valor);
13    }
14    return no;
15 }
16
17 No *buscarVar(No *no, const char *nome){
18     if(no == NULL){
19         return NULL;
20     }
21
22     int compare = strcmp(nome, no->nome);
23     if(compare == 0){
24         return no;
25     } else if (compare < 0){
26         return buscarVar(no->pEsq, nome);
27     } else{
28         return buscarVar(no->pDir, nome);
29     }
30 }

```

Código 2: Funções de inserção e busca na árvore

2.2.2 Gerenciamento da Pilha de Escopos

A pilha gerencia os escopos ativos durante a execução:

- adicionarPilha: cria um novo escopo e o empilha (comando begin)
- removerEscopo: desempilha o escopo atual, liberando sua memória (comando end)
- buscarVarPilha: busca uma variável percorrendo a pilha do topo até a base

```

1 No *buscarVarPilha(Pilha *pilha, const char *nome){
2     if(pilha == NULL){
3         return NULL;
4     }
5
6     Elemento *atual = pilha->pTopo;
7     while(atual){
8         if(atual->escopo){
9             No *var = buscarVar(atual->escopo->pRaiz, nome);
10            if(var){
11                return var;
12            }
13        }
14        atual = atual->pAnterior;
15    }
16
17    return NULL;
18 }

```

Código 3: Busca de variável na pilha de escopos

2.2.3 Processamento das Linhas

A função processarLinha interpreta cada comando do arquivo de entrada:

```

1 void processarLinha(Pilha *pilha, const char *linha){
2     char nomeVar[MAX_VAR];
3     char valorVar[MAX_VAR];
4
5     //ignora linhas vazias
6     if(strlen(linha) <= 1){
7         return;
8     }
9
10    //verifica inicio de escopo
11    if(strncmp(linha, "begin", 5) == 0){
12        adicionaPilha(pilha);
13        return;
14    }
15
16    //verifica declaracao de variavel
17    if(sscanf(linha, "var %s = %s", nomeVar, valorVar) == 2){
18        Escopo *atual = obterEscopoAtual(pilha);
19        if(!atual){
20            return;
21        }
22        adicionaVar(atual, nomeVar, valorVar);
23        return;
24    }
25
26    //verifica comando print
27    if(sscanf(linha, "print %s", nomeVar) == 1){
28        No *var = buscarVarPilha(pilha, nomeVar);
29        if(var){
30            printf("%s\n", var->valor);
31        } else{
32            printf("Variavel %s nao declarada\n", nomeVar);
33            exit(1);
34        }
35        return;
36    }
37
38    //verifica fim de escopo
39    if(strncmp(linha, "end", 3) == 0){
40        if(removerEscopo(pilha) == 0){
41            printf("Escopo nao aberto\n");
42        }
43        return;
44    }
45 }

```

Código 4: Processamento dos comandos

2.3 Decisões de Implementação

Algumas decisões importantes foram tomadas durante o desenvolvimento:

- **Pilha dinâmica:** a pilha foi implementada como uma lista encadeada, permitindo crescimento dinâmico e facilitando a liberação de memória.
- **Limitações de tamanho:** foram definidas constantes `MAX_LINE` e `MAX_VAR` para evitar estouro de buffer, assumindo que as linhas e nomes de variáveis não ultrapassem esses limites

3 Experimentos

Para avaliar o funcionamento do programa, foram realizados testes com diferentes arquivos de entrada, variando a complexidade dos aninhamentos e o número de variáveis declaradas. Os experimentos buscam verificar:

- *Correto gerenciamento de escopos aninhados*
- *Busca correta de variáveis em escopos externos*
- *Liberação correta de memória (verificado com Valgrind)*
- *Tempo de processamento para diferentes quantidades de comandos*

Os testes foram executados na máquina descrita na Seção 1.4, utilizando o Valgrind para verificar vazamento de memória e o GCC para análise de desempenho.

4 Resultados

Os testes realizados demonstram que o programa funciona corretamente para os casos previstos. A implementação com árvore binária mostrou-se adequada para o gerenciamento das variáveis, com tempos de busca satisfatórios mesmo para escopos com muitas variáveis.

4.1 Exemplos de Execução

Para o seguinte arquivo de entrada:

```
begin
var x = 2
var z = 7.5
begin
print x
print y
end
print z
end
```

A saída obtida foi:

```
2
Variavel y nao declarada
```

Este resultado está correto, pois a variável y tentou ser impressa sem antes ser declarada.

4.2 Análise de Memória

A execução com Valgrind não foi completamente satisfatória já que apontou Falha de Segmentação.

```
==11007== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
Falha de segmentação (imagem do núcleo gravada)
```

4.3 Pontos Fortes e Fracos

Pontos fortes:

- *Implementação clara e modular*
- *Busca eficiente de variáveis usando árvore binária*

Pontos fracos:

- *Gerenciamento incorreto de memória, com falha de segmentação*
- *Tratamento limitado de erros*

5 Considerações finais

O trabalho permitiu aplicar conceitos fundamentais de estruturas de dados (pilhas e árvores binárias) e gerenciamento de memória em C. A implementação de um interpretador simples para escopos aninhados demonstra na prática como linguagens de programação gerenciam variáveis em diferentes níveis de bloco. As principais dificuldades encontradas foram:

- *Garantir a correta liberação de memória em todos os casos, especialmente em situações de erro*
- *Implementar a busca correta de variáveis respeitando a ordem de escopos*
- *Tratar adequadamente os diferentes comandos do arquivo de entrada*

O conhecimento adquirido sobre gerenciamento de memória, estruturas de dados e interpretação de comandos será valioso para disciplinas futuras como compiladores e linguagens de programação.

Referências